

UNIVERSIDAD DEL ROSARIO



**DESARROLLO WEB PARA GESTIÓN E INVENTARIO PARA FERRETERIA
FERROALAMANO**

**Jeronimo Arguello Quintero
Daniel Alejandro Moreno Bravo**

20 de abril de 2026

TABLA DE CONTENIDOS

RESUMEN EJECUTIVO	3
CONTEXTUALIZACIÓN	4
FORMULACIÓN	4
OBJETIVOS	4
ALCANCE	5
METODOLOGÍA ÁGIL	5
PROPUESTAS DE SOLUCIÓN	5
RQF Y RQNF	6
MODELAMIENTO UML	11
PATRONES DE DISEÑO	19
DISEÑO DE INTERFACES	20
PRUEBAS DE USABILIDAD	24
ARQUITECTURA EMPRESARIAL	27
Misión, Visión y Objetivos Estratégicos	27
Capacidades de Negocio	28
Cadena de Valor del Negocio	28
Principios SOLID.	30
Patrón de Diseño: Singleton	30
DESIGN THINKING	30
Empatizar	30
Definir	31
Idear	32
Conclusiones del Ciclo de Vida del Desarrollo	33
REFERENCIAS	33

RESUMEN EJECUTIVO

El presente proyecto propone un desarrollo web para modernizar la ferretería Ferroalamano. Actualmente, este negocio local gestiona sus ventas, inventario y cotizaciones de forma completamente manual. El uso de cuadernos ha generado problemas operativos graves, como la venta de productos agotados, el cobro con precios desactualizados y la falta de control sobre las ganancias reales. Además, obligar a los clientes a pedir cotizaciones solo por teléfono o yendo al local retrasa el trabajo de contratistas y maestros de obra.

Para solucionar esto, se creará un sistema digital que centralizará todas estas tareas. La nueva plataforma permitirá a los vendedores facturar rápidamente, llevar un control exacto y automático de lo que hay en la bodega, y generar cotizaciones formales en formato PDF al instante.

El objetivo final de esta solución es eliminar los errores humanos y reducir el estrés del personal por la sobrecarga de trabajo. Con este cambio, el administrador tendrá información

clara y al día para tomar mejores decisiones , y los clientes disfrutarán de un servicio mucho más rápido y organizado para planificar sus construcciones.

CONTEXTUALIZACIÓN

La ferretería Ferroalamano es un negocio local que comercializa materiales de construcción, herramientas y accesorios para el hogar. Su clientela principal está compuesta por contratistas independientes, maestros de obra y compradores particulares. Actualmente, la ferretería no cuenta con ningún sistema digital para la gestión de sus operaciones internas, lo que genera ineficiencias en tres procesos: las cotizaciones de material, la gestión de inventario y el registro de ventas.

En cuanto al stock, el registro de existencias se ha llevado en cuadernos o papeles, lo que expone al negocio a errores frecuentes como ventas de productos agotados, pérdidas por desactualización de precios y dificultades para hacer seguimiento al stock real. Finalmente, la ausencia de un módulo de ventas impide llevar un control preciso sobre las transacciones realizadas, afectando la toma de decisiones del administrador.

FORMULACIÓN

En este contexto surge la siguiente pregunta problema: ¿Cómo desarrollar un sistema de información web que permita a la ferretería Ferroalamano centralizar y optimizar sus procesos de gestión de stock y registro de ventas, superando las limitaciones operativas de los procesos manuales actuales?

La digitalización de estos procesos mediante una aplicación web permitirá reducir errores humanos, mejorar la trazabilidad de las operaciones y ofrecer disponibilidad de información en tiempo real. Por otro lado, el proyecto impacta positivamente al personal de la ferretería, que verá reducida su carga operativa, y a los clientes quienes podrán obtener cotizaciones de forma más ágil y formal, facilitando la planificación de sus proyectos de construcción.

OBJETIVOS

Objetivo general: Desarrollar una aplicación web que centralice y optimice los procesos de registro de ventas y generación de cotizaciones de la ferretería Ferroalamano, mejorando la eficiencia operativa y la experiencia de sus usuarios.

Objetivos Específicos:

1. Implementar un módulo de venta que permita la facturación ágil y la búsqueda eficiente de productos.
2. Diseñar la base de datos no relacional para almacenar toda la información relacionada
3. Estructurar un gestor de cotizaciones que permita exportar documentos en PDF para contratistas

ALCANCE

Incluye: Aplicación web (Frontend y Backend), base de datos relacional o no relacional, stock, venta y Clientes.

Excluye: Integración con facturación electrónica, pasarelas de pago online (solo registra pagos físicos)

Público objetivo: Personal administrativo y vendedores de la ferretería Ferroalamano

METODOLOGÍA ÁGIL

Con el fin de tener un ritmo de trabajo adecuado se implementará un tablero kanban donde se llevará registro de todas las tareas que se van a realizar durante el desarrollo del proyecto:

<https://trello.com/invite/b/69aca8025231c5577a6c2468/ATTIde89e8b6a2b50ed41e613010dc4b0b399767DE11/kanban>

PROPUESTAS DE SOLUCIÓN

Con el fin de proponer una solución adecuada se investigó sobre la importancia de los desarrollos web para instituciones o empresas, encontrando evidencia que respalda su impacto positivo en negocios, especialmente en sectores de comercio y suministro de materiales. En primer lugar, estudios recientes señalan que las pequeñas empresas enfrentan retos críticos en la gestión de inventario, entre ellos la falta de visibilidad en tiempo real sobre los niveles de stock y la dependencia de procesos manuales propensos a errores, lo que compromete su capacidad de operar eficientemente y mantenerse competitivas [1]. Ante esto, la implementación de un sistema de inventario basado en web ha demostrado generar altos niveles de satisfacción entre los usuarios, con un 93.3 % expresando satisfacción, lo que valida la efectividad de este tipo de soluciones digitales para contextos como el de una ferretería [1]. En segundo lugar, la integración de sistemas de gestión de inventario ofrecen seguimiento en tiempo real de los niveles de existencias, alertas automatizadas de reorden y conexión con otros sistemas del negocio, mejorando la eficiencia operativa. Un caso de estudio en una empresa de logística demostró que implementar un sistema de este tipo redujo los costos de almacenamiento en un 24 % [2]. Finalmente, investigaciones orientadas específicamente al sector de suministro de materiales de construcción han identificado que los pequeños comercios de este nicho utilizan frecuentemente sistemas de gestión de inventario desactualizados, y proponen que la creación de sistemas web centrados en el usuario constituye una vía concreta para reducir la brecha tecnológica y modernizar este tipo de negocios [3].

Ahora, teniendo en cuenta la investigación realizada, se propone la siguiente solución. se plantea el desarrollo de una aplicación web completa para la ferretería Ferroalamano bajo una arquitectura cliente-servidor, donde el frontend y el backend operan como sistemas independientes que se comunican mediante el HTTP y el manejo de datos en formato JSON.

Se busca implementar con MongoDB, una base de datos orientada a documentos que almacena la información en estructuras flexibles similares, sin requerir un formato fijo. Esta arquitectura responde directamente a las tres necesidades centrales de Ferroalamano. Por otro lado se proponen las siguientes clases:

- Usuario
- Cliente
- Venta
- Factura
- DetalleFactura
- Producto

RQF Y RQNF

Ya con una solución definida se proponen los siguientes requerimientos funcionales:

CÓDIGO	REQUISITO FUNCIONAL
RQF001	Nombre: Iniciar sesión Descripción: El sistema permitirá a los usuarios loguearse con credenciales válidas para acceder a los módulos según sus permisos. Usuarios: administrador, vendedor
RQF002	Nombre: Buscar Producto por SKU Descripción: El sistema permitirá buscar rápidamente un producto ingresando su código único SKU. Usuarios: admin, vendedor

RQF003	Nombre: Registrar Nuevo Producto Descripción: El sistema permitirá crear un nuevo ítem en el inventario, asignándole un nombre, descripción y código SKU. Usuarios: administrador
RQF004	Nombre: Asignar Categoría Descripción: El sistema permitirá clasificar el producto dentro de categorías predefinidas (ej. Eléctricos, Plomería, Herramientas). Usuarios: administrador

Para ver los demás RQF acceder al siguiente link:

<https://github.com/Danialej0/ProyectoArquitecturaSoftware/blob/f33c60c145e28f0236df0231336ec59507af6851/primer%20entegable/ANEXO%2C%20RQF%2C%20RQNF%2C%20HU.pdf>

En cuanto a los RQNF se plantean los siguientes:

CÓDIGO	REQUISITO NO FUNCIONAL
RQNF001	Nombre: Diseño de UI Descripción: El sistema debe contar con una interfaz intuitiva y fácil de usar, permitiendo que los usuarios puedan navegar sin dificultades. Usuarios: administrador, vendedor

RQNF002	Nombre: Implementar sistema de seguridad en la plataforma Descripción: Se deben implementar mecanismos de autenticación y autorización para proteger la información de toda la empresa Usuarios: admin, vendedor
RQNF003	Nombre: Optimizar el tiempo de respuesta Descripción: Asegurar que las consultas y funcionalidades de las bases de datos, tanto relacional como no relacional, se ejecuten en máximo 2 segundos. Usuarios: administrador, vendedor
RQNF004	Nombre: Portabilidad del Sistema Descripción: El sistema deberá poder ser desplegado en cualquier proveedor de servicios que soporte Node.js y MongoDB Usuarios: administrador, vendedor

Con los RQF ya definidos es pertinente realizar las historias de usuario, nuevamente, para ver las HU de manera completa acceder al siguiente link:

<https://github.com/Danialej0/ProyectoArquitecturaSoftware/blob/f33c60c145e28f0236df0231336ec59507af6851/primer%20entegable/ANEXO%20casos%20de%20uso.pdf>

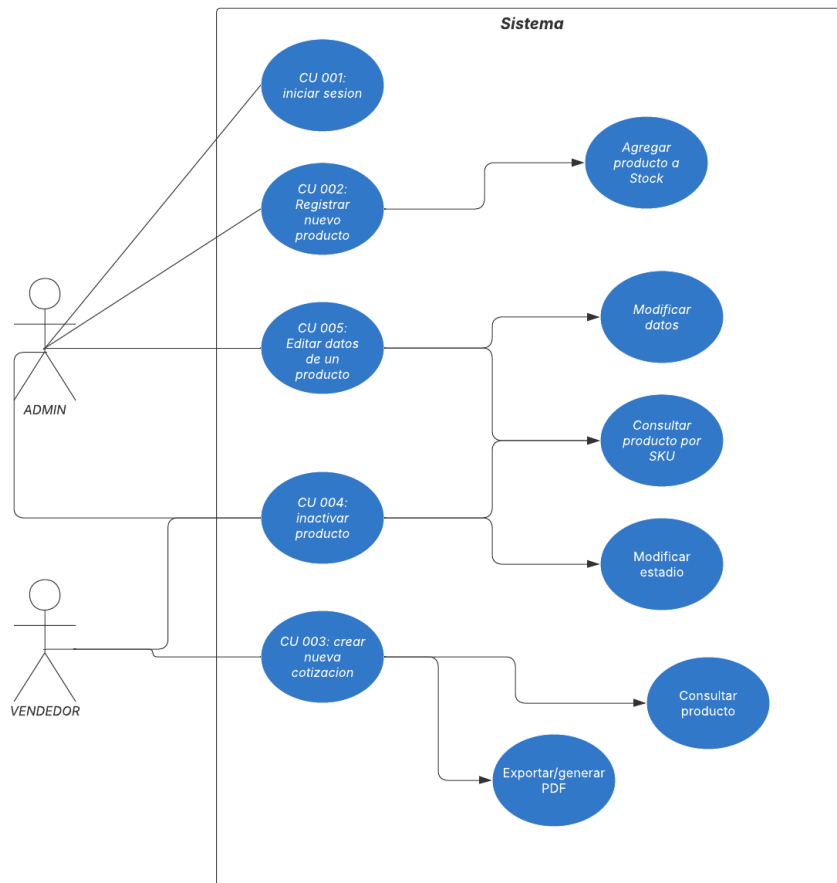
CÓDIGO	HISTORIA DE USUARIO
--------	---------------------

HU001	Nombre: Iniciar Sesión Descripción: Como Administrador o Vendedor, quiero iniciar sesión con mis credenciales para poder acceder a los módulos de la plataforma según los permisos asignados a mi rol Criterios de Aceptación: - Validar las credenciales ingresadas - Restringir solo para máximo tres intentos fallidos. - Bloquear la cuenta si se cumple la condición de los tres intentos fallidos - Permitir recuperar la cuenta en caso de olvidar algún dato.
HU002	Nombre: Buscar Producto por SKU Descripción: como Administrador o Vendedor, quiero buscar un producto ingresando su código SKU para localizarlo de forma rápida y precisa sin tener que navegar por todo el catálogo Criterios de Aceptación: - Mostrar el producto correspondiente al SKU ingresado si existe en el sistema. - Mostrar un mensaje indicativo si no se encuentra ningún producto asociado al SKU ingresado. - Mostrar en el resultado el nombre, categoría, precio y stock disponible del producto encontrado. Puntos de Esfuerzo: 2
HU003	Nombre: Registrar Nuevo Producto Descripción: Como administrador, quiero registrar un nuevo producto en el sistema para mantener el catálogo de inventario actualizado con todos los artículos disponibles en la ferretería Criterios de aceptación: - ingresar obligatoriamente el nombre,

	descripción y código SKU del producto - Validar que el código SKU ingresado no esté previamente registrado en el sistema, mostrando un error si ya existe. - Confirmar el registro exitoso del producto mediante un mensaje de éxito visible al usuario. - Puntos de Esfuerzo: 3
HU004	Nombre: Asignar Categoría a Producto Descripción: como Administrador, quiero clasificar cada producto dentro de una categoría predefinida para organizar el catálogo y facilitar la búsqueda Criterios de Aceptación: - Mostrar un listado de categorías predefinidas disponibles (Eléctricos, Plomería, Herramientas, entre otras) al momento de crear o editar un producto - Guardar la categoría asignada junto con la información del producto de forma correcta en la base de datos - Reflejar la categoría asignada en el listado de productos y en los resultados de búsqueda
HU005	Nombre: Crear Nueva Cotización Descripción: Como vendedor o administrador, quiero crear una nueva cotización para generar un documento formal con los productos y precios requeridos por un cliente o contratista. Criterios de Aceptación: - Permitir agregar uno o más productos a la cotización - Asignar automáticamente un número único a cada cotización creada. - Guardar la cotización con estado "Pendiente" por defecto al momento

MODELAMIENTO UML

Para iniciar con el modelamiento UML del desarrollo es necesario realizar los casos de uso con su respectivo diagrama. En este caso se realizaron 5 casos de usos.



De igual forma se realizó la documentación:

1 IDENTIFICACIÓN DE CASO DE USO			
1.1 Id Caso	CU D 002	1.2 Nombre	Registrar Nuevo Producto
2 HISTÓRICO DE CASO DE USO			

2.1 Autor	Daniel Moreno Bravo		
2.2 Fecha Creación	5/03/26	2.3 Fecha última Actualización	5/03/26
2.4 Actualizado por	Daniel Moreno	2.5 Versión	1.0

3 DEFINICIÓN DE UN CASO DE USO

3.1 DESCRIPCIÓN

Mediante este caso de uso el administrador podrá crear un nuevo ítem en el catálogo de inventario, asignándole nombre, descripción, código SKU, categoría, precio, unidad de medida y stock mínimo.

3.2 ACTORES

Administrador

3.3 PRECONDICIONES

El administrador debe haber iniciado sesión en el sistema. Las categorías predefinidas deben existir en la base de datos.

3.4 FLUJO NORMAL

Contando con las precondiciones el flujo normal será el siguiente:

Pas o	Actor	Sistema
1	Navega al módulo de Inventario y selecciona 'Registrar Producto'	El sistema muestra el formulario de registro con los campos: nombre, descripción, SKU, categoría (lista desplegable), precio, unidad de medida y stock mínimo.
2	Diligencia todos los campos obligatorios y selecciona la categoría correspondiente	El sistema valida en tiempo real que el SKU ingresado no exista previamente en la base de datos.
3	Confirma el registro haciendo clic en 'Guardar'	El sistema almacena el nuevo producto en la base de datos y muestra el mensaje: "Producto registrado exitosamente". El producto queda disponible en el catálogo.

3.5 FLUJO ALTERNATIVO

Si existe otra forma de acceder al caso de uso, describir los pasos.

Pas o	Actor	Sistema
2.a	Hace clic en el botón 'Cancelar'	El sistema descarta los datos ingresados y redirige al listado general de productos sin realizar ningún cambio en la base de datos.

3.6 FLUJO EXCEPCIONAL

Pas o	Actor	Sistema
----------	-------	---------

2.a	Ingresa un código SKU que ya existe en el sistema	El sistema muestra el mensaje: "El código SKU ingresado ya está registrado. Por favor ingrese un código único" y resalta el campo en rojo.
2.b	Deja campos obligatorios vacíos e intenta guardar	El sistema muestra el mensaje: "Debe completar todos los campos obligatorios" e impide guardar hasta que se corrijan los errores.

3.7 POS CONDICIONES	
Sistema permite el registro de un nuevo producto, añadiéndolo a la base de datos.	
3.8 FRECUENCIA	
media	

Para ver la documentación completa, acceder al repositorio de proyecto.

Después de haber realizado el levantamiento de información y de haber determinado los objetivos y el alcance del proyecto, era necesario establecer cuales son los stakeholder del proyecto. “Los stakeholders del proyecto son las partes interesadas que pueden influir o verse afectadas por el proyecto en el que estás trabajando. Los participantes pueden provenir de todos los niveles de la organización, desde colaboradores individuales hasta ejecutivos” (Martins, 2025). Ya con una definición adecuada, se llegó al siguiente mapa de Stakeholders:

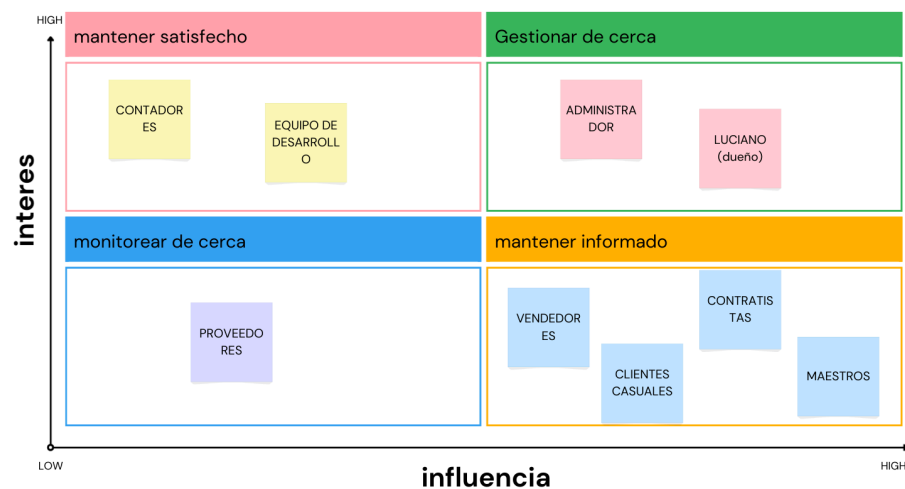
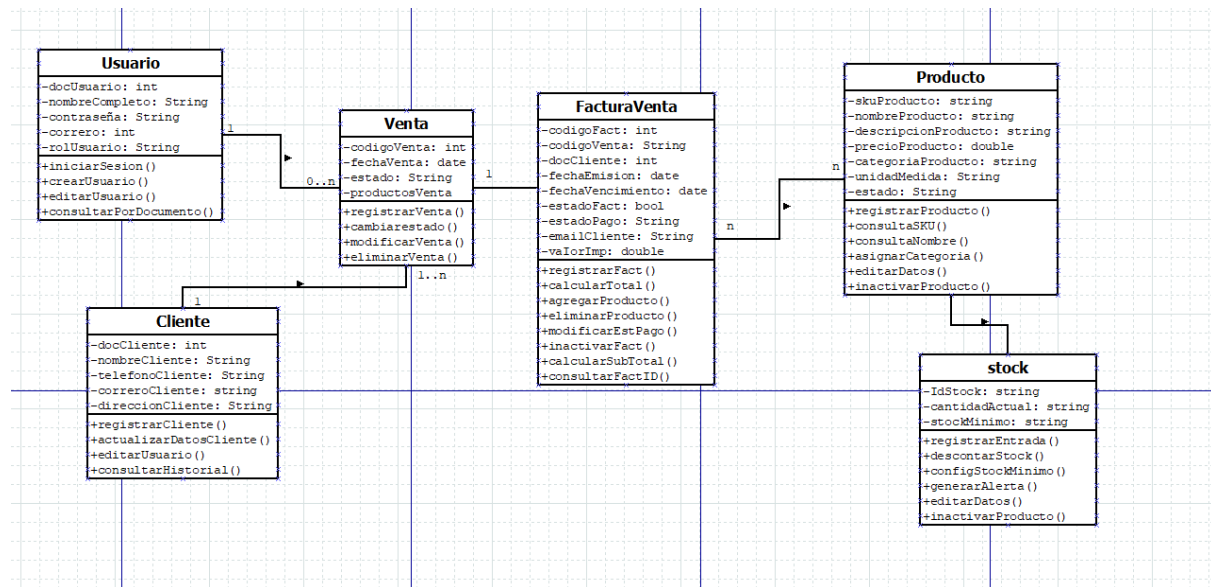


DIAGRAMA DE CLASES:



A continuación, se presenta la interpretación del diagrama de clases para el proyecto:

La clase Usuario, con los atributos docUsuario, nombreCompleto, correoUsu, contraseñaUsu y rolUsu; donde su atributo clave es docUsuario. Cuenta con las operaciones iniciarSesion(), crearUsuario(), editarUsuario() y consultarPorDocumento().

La clase Cliente, con los atributos docCliente, nombreCliente, telefonoCliente y direccionCliente; donde el atributo clave es docCliente. También, cuenta con los métodos de registrarCliente(), validarDocumento(), actualizarDatos() y consultarHistorial().

La clase Venta contiene los atributos codigoVenta, fechaTransaccion y estadoVenta; donde el atributo clave es el codigoVenta. Sus métodos son registrarVenta(), facturarCotizacion() y cancelarVenta().

Relaciones: Se relaciona mediante asociación donde 1 Cliente puede tener 0 a n (múltiples) Ventas. Asimismo, 1 Usuario se relaciona con Venta donde un usuario puede gestionar 0 a n ventas.

La clase Factura, con los atributos codigoFactura, fechaEmision, subtotalFact, impuestosFact y totalFinal; donde el atributo clave es codigoFactura. Sus operaciones son generarFactura() y calcularTotal().

Relaciones: Se relaciona mediante asociación directa donde 1 Venta genera exactamente 1 Factura.

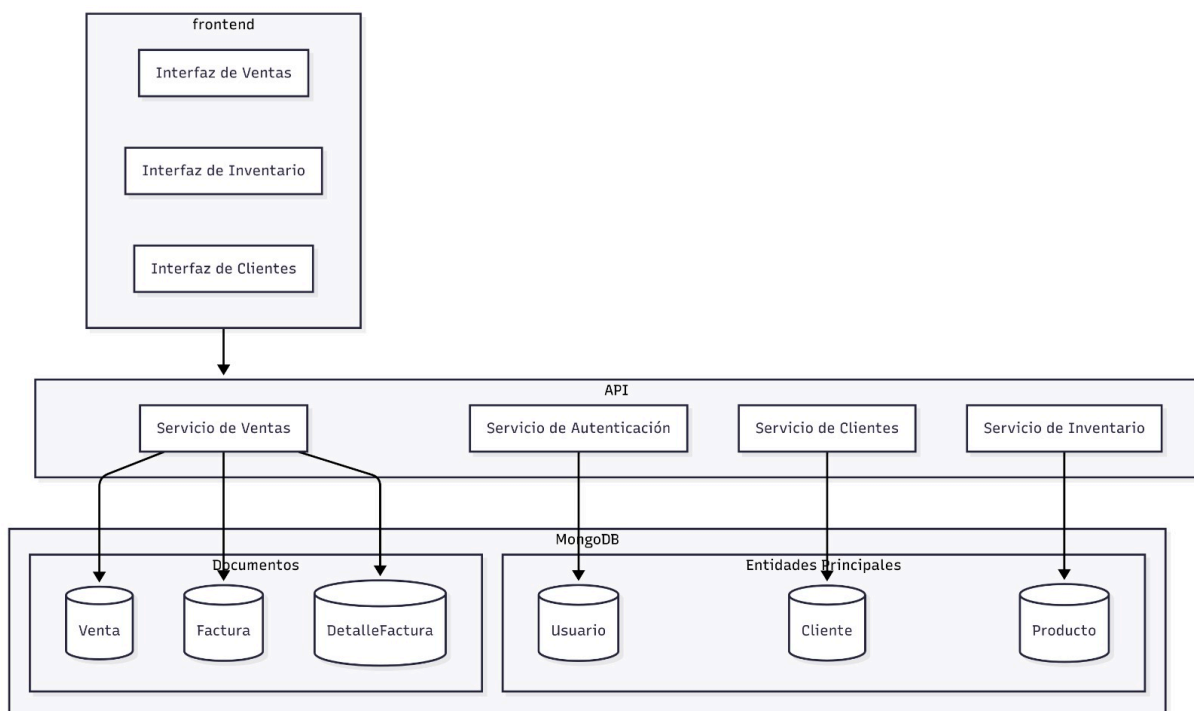
La clase Producto, con los atributos skuProducto, nombreProducto, descripProducto, precioProducto, categoriaProd, unidadMedida y estadoActivo; donde su atributo clave es skuProducto. Sus métodos son registrarProducto(), buscarPorSKU(), buscarPorNombre(),

asignarCategoria(), definirUnidadMedida(), editarDatos(), consultarPorPrecio() e inactivarProducto().

Relaciones: Se relaciona con Stock donde 1 Producto tiene exactamente 1 registro de Stock.

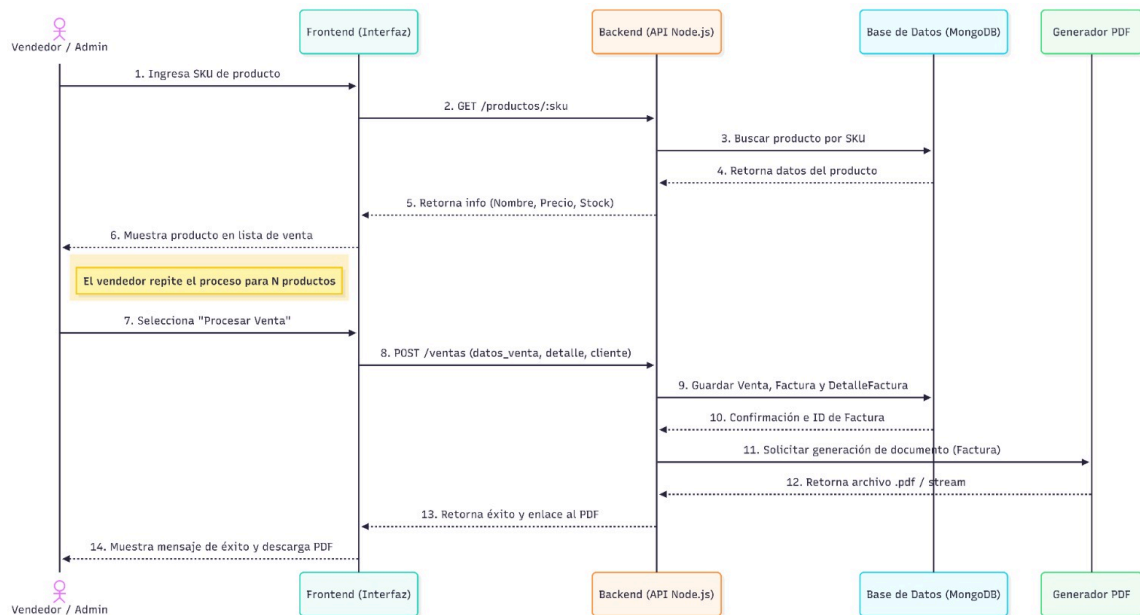
La clase Stock, con los atributos idStock, cantidadActual y stockMinimo; donde el atributo clave es idStock. Sus operaciones son registrarEntrada(), descontarStock(), configurarStockMinimo(), generarAlerta() y consultarInventario().

- DIAGRAMA DE PAQUETES



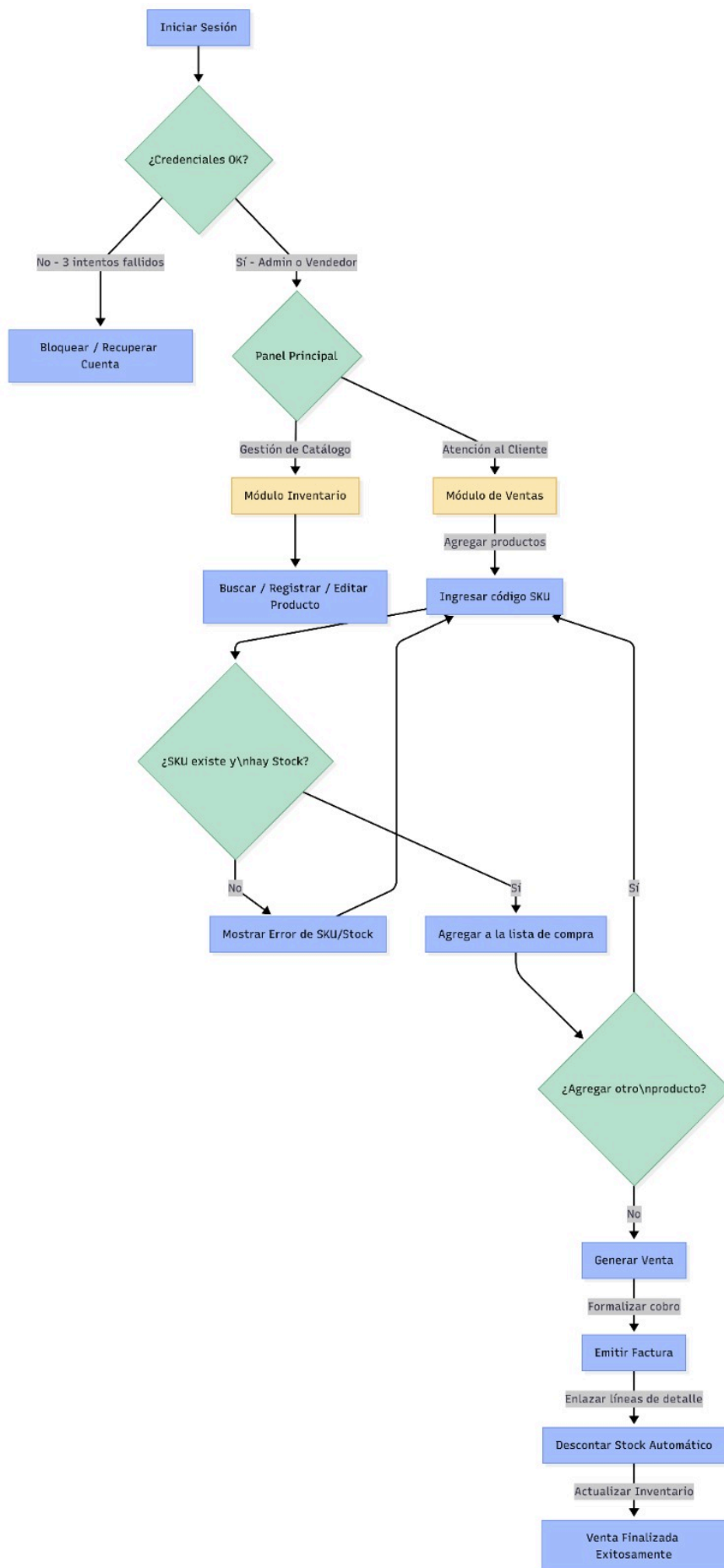
El diagrama de paquetes del sistema Ferroalamano organiza la arquitectura en tres capas horizontales. La capa superior agrupa los cinco paquetes de interfaz de usuario (UI), uno por cada módulo funcional: autenticación o log in, Inventario, Ventas, Cotizaciones y Clientes; cada uno de estos se conecta mediante una relación «use» hacia su servicio correspondiente en la capa intermedia, indicando que la interfaz delega toda la lógica de negocio al backend. En la capa de Servicios, VentaService mantiene relaciones «access» con ProductoService y CotizacionService, dado que al facturar una venta se requiere descontar stock y convertir una cotización aprobada; a su vez, CotizacionService tiene una relación «access» con ClienteService porque toda cotización debe estar asociada a un cliente registrado. Todos los servicios comparten una única instancia de ConexionBD mediante una relación «access», reflejando el patrón Singleton que garantiza una sola conexión activa a la base de datos durante toda la ejecución del sistema. Finalmente, cada servicio importa su modelo r a través de relaciones «import», y ConexionBD importa el paquete BaseDatos MongoDB, que es donde residen las colecciones del sistema.

-DIAGRAMA DE SECUENCIAS



Una vez el vendedor ha seleccionado todos los materiales necesarios, el sistema se encarga de agrupar esta información para guardar la venta y su facturación un número único y en estado pendiente. Finalmente, el diagrama muestra el cierre del proceso, donde el servidor solicita al generador de documentos la creación de un archivo PDF que el vendedor podrá descargar de inmediato. En resumen, este diagrama nos asegura que desde la búsqueda inicial hasta la entrega del documento final, cada pieza del software de Ferroalamano trabaje de forma coordinada para eliminar las demoras de los procesos manuales actuales

-DIAGRAMA DE ESTADOS



El proceso comienza con el inicio de sesión del usuario; si hay tres intentos fallidos, la cuenta se bloquea y debe recuperarse, pero si las credenciales son correctas, se accede al Panel Principal, el cual se divide en dos áreas operativas. Por un lado, el Módulo de productos permite gestionar el catálogo buscando productos por su Id, editándolos o inactivándolos, y otorga a los administradores la capacidad de registrar nuevos artículos tras validar que su código no exista previamente en el sistema para luego guardarlos en la base de datos. Por otro lado, el Módulo de Ventas facilita la atención al cliente al permitir crear presupuestos, asignarles un número único, guardarlos como pendientes y exportarlos en PDF donde se emite la factura y el sistema descuenta los artículos automáticamente para mantener el stock actualizado.

PATRONES DE DISEÑO

Para garantizar que el código sea mantenible, escalable y eficiente, se propone la implementación de los siguientes tres patrones de diseño en el desarrollo de la aplicación web:

1. Patrón Singleton

Justificación: Como se mencionó en el diagrama de paquetes, es fundamental garantizar que exista una única instancia de conexión a la base de datos (MongoDB) durante toda la ejecución del sistema.

Aplicación: Se creará una clase ConexionBD que restrinja la instanciación de la conexión. Si un servicio (como Producto o Venta) necesita interactuar con la base de datos, solicitará la instancia a esta clase. Si la instancia ya existe, se devuelve la misma; si no, se crea una nueva. Esto evita la saturación de conexiones y fugas de memoria.

ESTILOS ARQUITECTÓNICOS

Arquitectura Cliente-Servidor basada en API

Para solucionar las ineficiencias operativas de Ferroalamano, se ha seleccionado un estilo arquitectónico Cliente-Servidor. Esta decisión se alinea con la propuesta de tener el frontend (interfaz de usuario) y el backend (lógica y datos) operando como sistemas independientes y desacoplados.

Capa de Frontend: Será una aplicación web responsiva, renderizada en el navegador del usuario (Administrador o Vendedor). Se encargará exclusivamente de la interfaz gráfica, la validación de formularios en el lado del cliente y la experiencia de usuario.

Capa de Lógica (Backend): Construida con Node.js, actuará como una API. Recibirá las peticiones HTTP del cliente, procesará la lógica (ej. calcular totales de factura, generar PDF de cotizaciones, validar credenciales y gestionar las reglas de negocio).

Capa de Datos: Una base de datos no relacional MongoDB, que se comunicará exclusivamente con el Backend. Toda la transferencia de datos entre el Cliente y el Servidor se realizará mediante el formato ligero JSON.

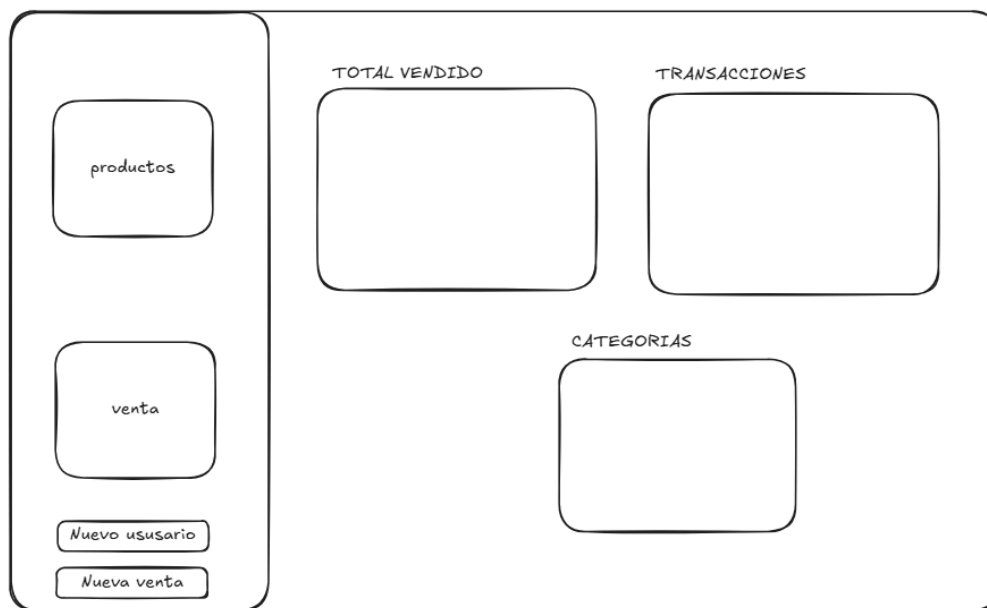
DISEÑO DE INTERFACES

Ya teniendo clara la estructura del proyecto, es necesario empezar a plantear los diseños para el posterior desarrollo. En un primer momento se plantearon los siguientes wireframes para las pantallas principales del desarrollo, como por ejemplo:

Wireframe para el inicio de sesión:

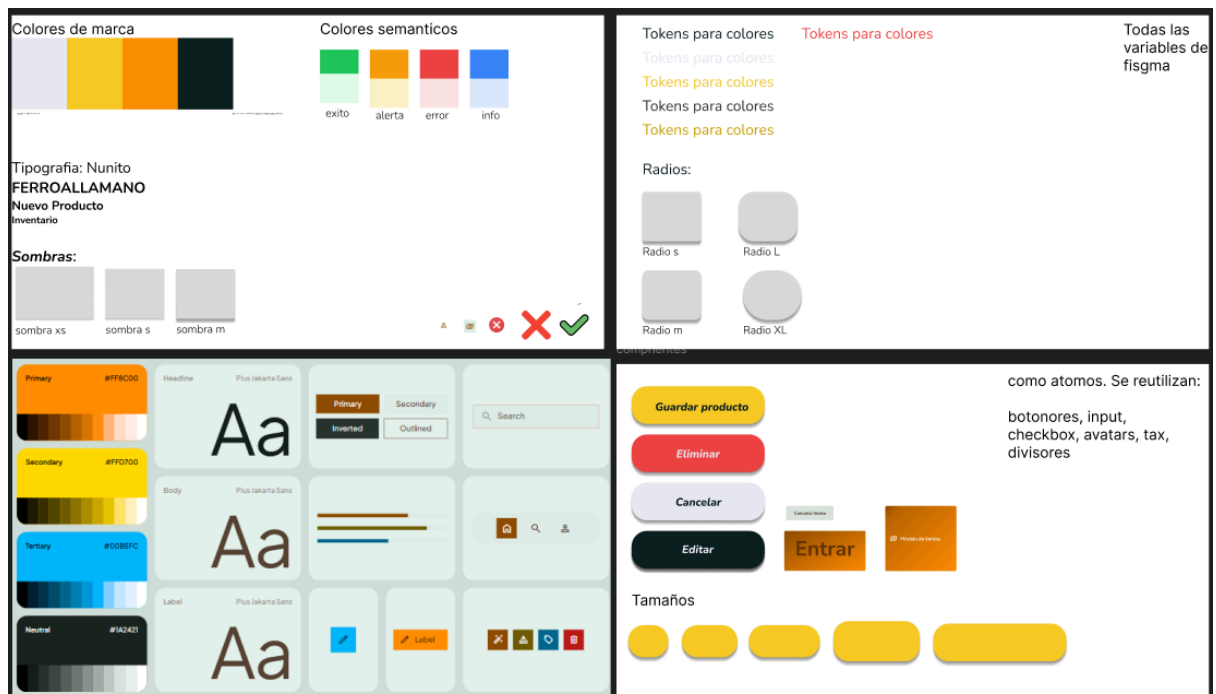
The wireframe is a vertical rectangle with rounded corners. At the top, the text 'FERROALAMANO' is centered in a bold, sans-serif font. Below it, 'INICIAR SESION' is centered in a smaller, all-caps, sans-serif font. Further down, the label 'USUARIO' is centered above a rectangular input field. Below that, the label 'CONTRASEÑA' is centered above another rectangular input field. Under the password field, there is a small square checkbox followed by the text 'OLVIDE CONTRASEÑA'. At the bottom, a rectangular button is centered with the text 'ENTRAR'.

Wireframe del dashboard:

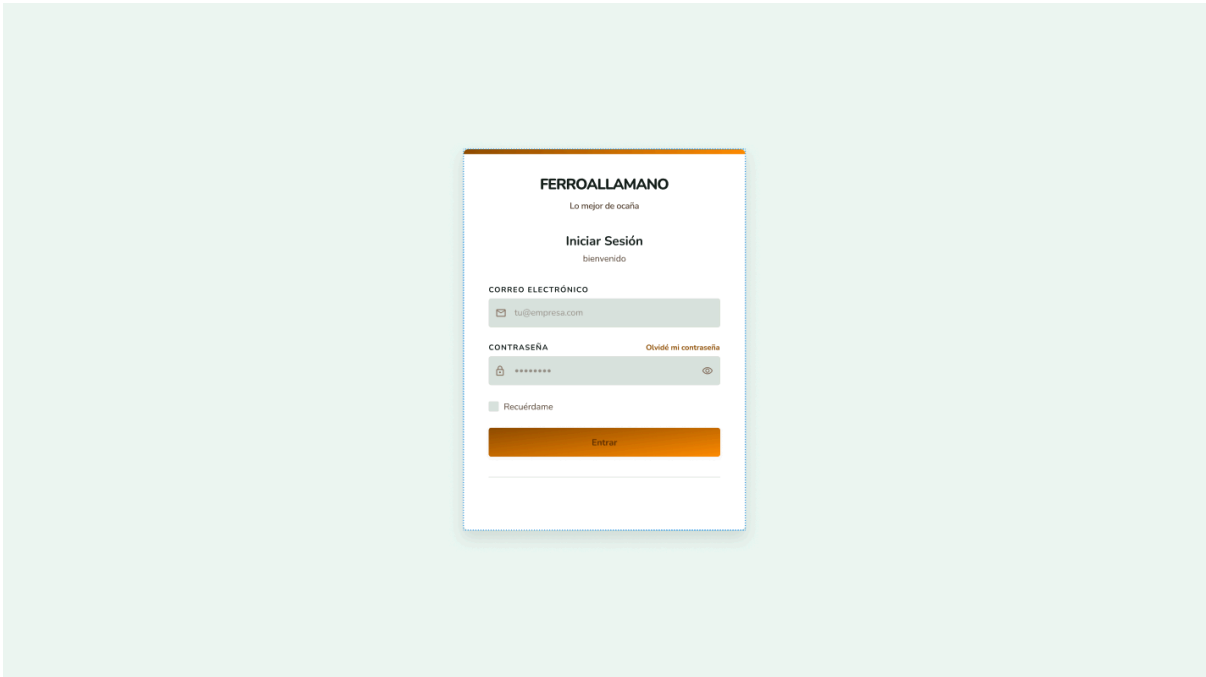


y demás wireframes requeridos para realizar todo el apartado visual del proyecto. Dentro del repositorio se encuentran todas las pantallas planteadas.

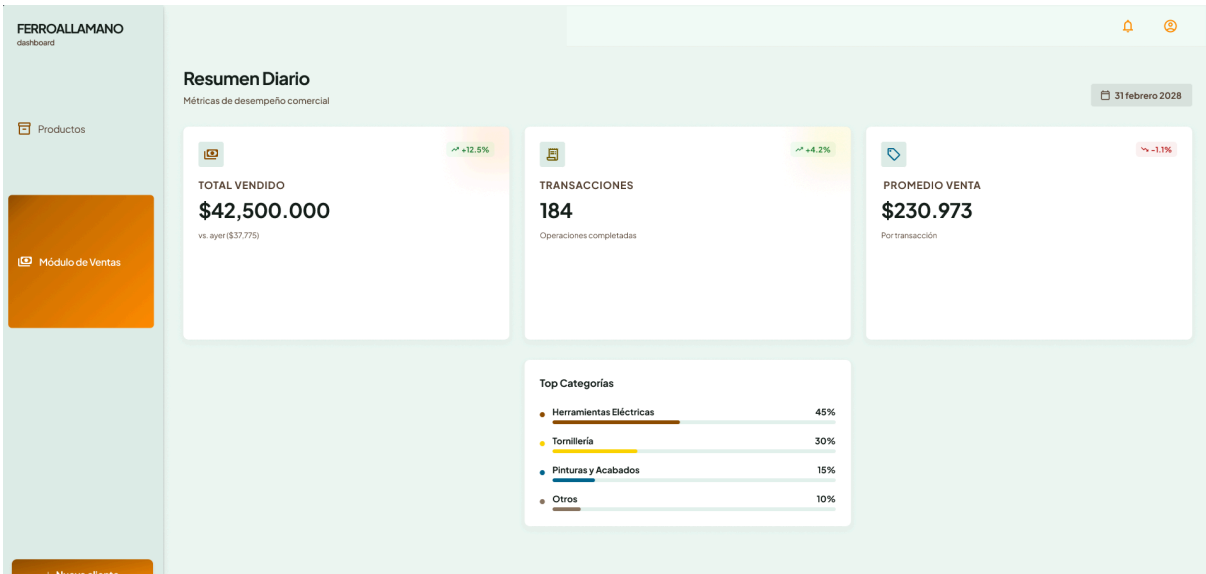
Con todos estos modelos de baja fidelidad ya planteados se procedió a establecer un UI kit pertinente para el desarrollo, teniendo en cuenta la teoría del color y el contexto de la ferretería. Obteniendo así lo siguiente:



Basándose en el presente UI kit y teniendo en cuenta todos los lineamientos de UI/UX, se procedió a diseñar los prototipos de Alta fidelidad mediante la herramienta de figma. Algunas de las pantallas son las siguientes:



Se puede apreciar claramente los principios de contraste y consistencia dentro de la primera pantalla de la aplicación.



Productos

Módulo de Ventas

Dashboard

Nueva Venta

VEN-2028-089 31 febrero, 2028

Complete los detalles para registrar una nueva transacción comercial.

Información del Cliente

Buscar Cliente

Nuevo Cliente

Detalle de Productos

Ingresar SKU o Nombre de producto

SKU	PRODUCTO	CANTIDAD	PRECIO UNIT.	SUBTOTAL
ACER-001	Varilla de acero Dim: 2" x 1/4" x 6m	- 12 +	\$ 450.500	\$ 546.00
SOLD-045	Electrodo E6011 Lincoln Caja x Skg - 1/8"	- 3 +	\$ 18.205	\$ 54.705

Notas de Venta

Observaciones adicionales...

Subtotal:

\$ 600.75

IVA (19%):

\$ 72.09

Descuento:

-\$ 0.00

Total:

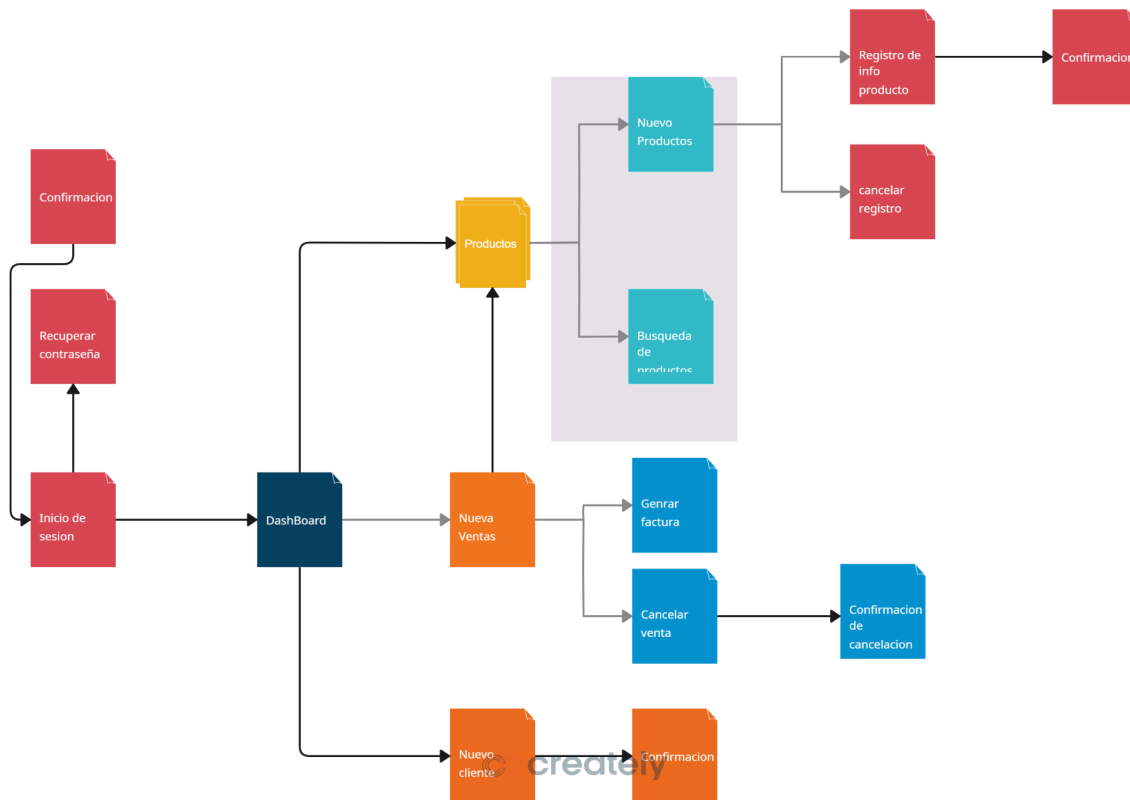
\$ 672.84

Cancelar Venta

Registrar Venta

Estos son algunos ejemplos del Prototipo de alta fidelidad del proyecto, para consultar todas las pantallas, nuevamente, se sugiere entrar al repositorio o acceder al siguiente link: <https://www.figma.com/proto/XbZqnsM3USWcKUQrXiGw6Z/FerroAllamano?node-id=92-1947&p=f&t=Z4u174h69iV9APNG-0&scaling=min-zoom&content-scaling=fixed&page-id=20%3A100&starting-point-node-id=40%3A88>

Al momento de realizar el modelo hifi también es necesario realizar el mapa de navegación.



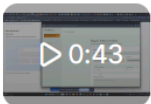
Se buscó , mediante los principios de UX que la navegación dentro del desarrollo sea amigable, que se puedan acceder de manera intuitiva a cualquier lo que se evidencia en las siguientes pruebas de usabilidad.

PRUEBAS DE USABILIDAD

Para probar la calidad del diseño planteado se propusieron las siguientes tareas en MAZE:

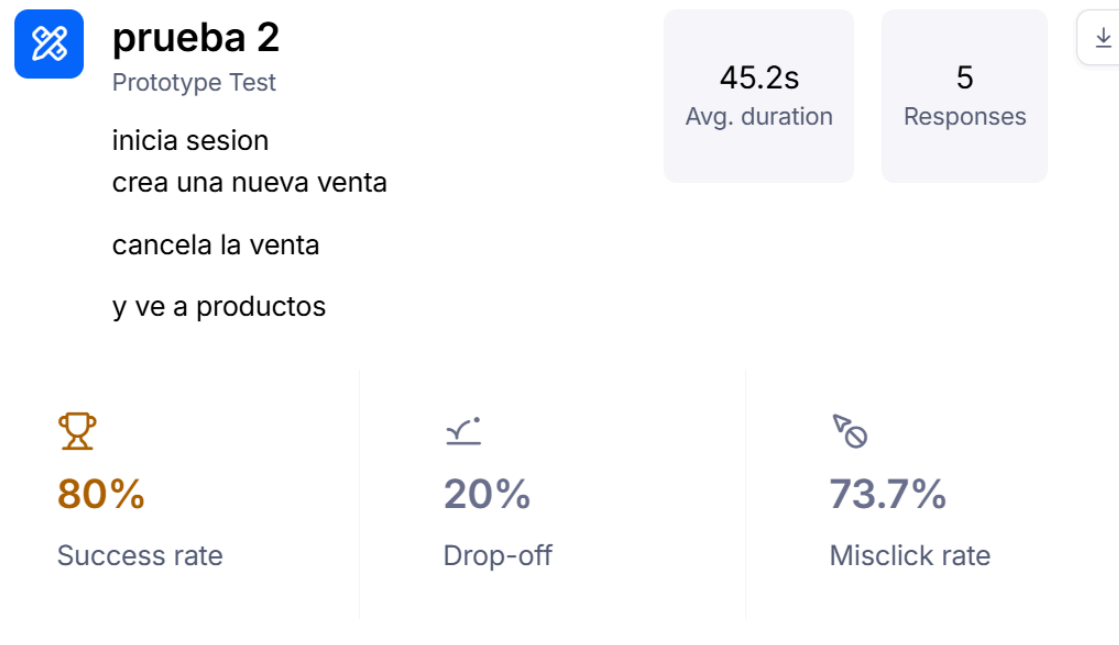
- Prueba 1: Deberá iniciar sesión y agregar un nuevo producto y luego volver al dashboard.

De esta prueba se obtuvieron los siguientes resultados:

CLIPS	PARTICIPANT	DEVICE	DURATION	RESPONDED AT	ACTIONS
	524883500		36.28s	19 Apr 2026, 07:22 pm	
	524880376		38.82s	19 Apr 2026, 06:56 pm	

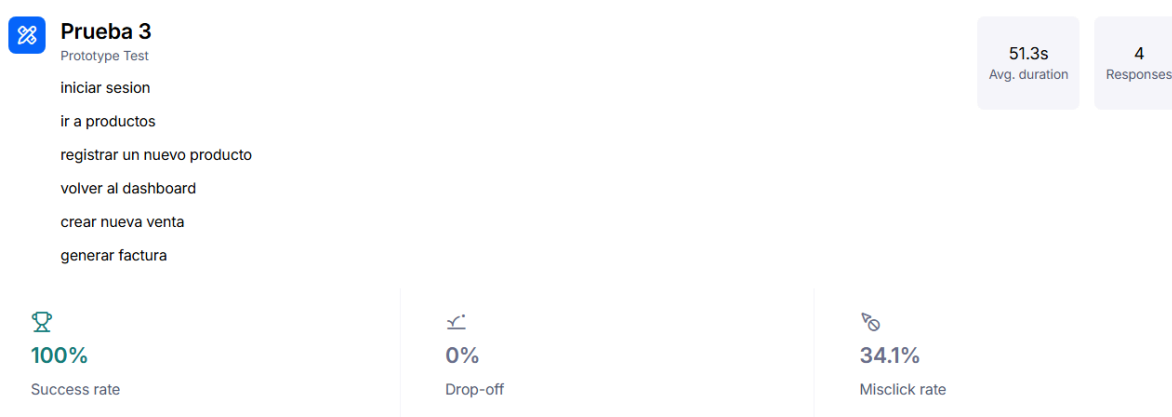
Se observa que esta tarea un un succes rate del 100% con un tiempo inferior a los 40 segundos indicando que personas sin conocimiento previo de la aplicación fueron capaces de entender el UI del desarrollo de las primeras pantallas.

- Prueba 2: inicia sesión, crea una nueva venta, cancela la venta y ve a productos



Dentro de esta tarea se planteó una ruta específica de lo que se obtuvo los siguientes mapas de calor para cada una de las pantallas:

- Prueba 3: iniciar sesion, ir a nuevo cliente, registrar un nuevo cliente, volver al dashboard, crear nueva venta, generar factura



Para resumir los resultados obtenidos de estas pruebas se creó la siguiente tabla:

Prueba	Flujo	Usuarios	Duración Prom.	Success Rate	Drop-off	Drop-off
P1	Login → Nuevo Producto → Dashboard	2	~37s	100%	0%	N/D
P2	Login → Nueva Venta → Cancelar → Productos	5	45.2s	80%	20%	73.7%
P3	Login → Productos → Nueva Venta → Factura	4	51.3s	100%	0%	34.1%

De estas pruebas se pudo observar lo siguiente.

- Dentro de la pantalla del dashboard al realizar las pruebas 2 y 3 se observan que los clics son dispersos al momento de buscar la funcionalidad deseada como lo son, “nueva venta”, “productos” y “nuevo cliente” los usuarios exploran el dashboard antes de encontrar el acceso a productos. Dentro de los participantes de las pruebas se sugirió cambiar de lugar los botones “nueva venta” y “nuevo cliente”, además de aumentar su tamaño, con el fin de que sean más sencillos de encontrar.
- Dentro de la pantalla de “nueva venta” se sugirió espaciar los botones de “cancelar venta” y “generar factura”. Si bien se pensó en una pantalla para la confirmación de cancelaciones, el señor fabio realizó esta recomendación.
- Por otro lado, se sugirió cambiar la resolución de todas las pantallas, pues al momento de crear el modelo de alta fidelidad se estableció que 1920x1080px fuese el tamaño estándar, resolución que no permite ver todo el contenido sin necesidad de “scrollear” lo que impide ver todo el contenido del desarrollo.

Por tanto, se busca mejorar lo siguiente:

- El 73.7% de missclicks es un indicador crítico. Se busca ajustar la jerarquía visual del flujo "Nueva Venta → Cancelar → Productos", aumentar el tamaño y contraste de los elementos clave y reducir elementos clickables innecesarios en esas pantallas.

- Los mapas de calor muestran que los usuarios exploran las métricas del Resumen Diario en búsqueda de navegación. Considerar separar visualmente las zonas de datos informativos

En términos generales, se cuenta con un buen nivel de satisfacción del usuario al momento de interactuar con el desarrollo. Se resaltó la paleta de colores y los iconos empleados.

Para ver los mapas de calor completos correspondientes a cada prueba, se recomienda revisar el repositorio:

ARQUITECTURA EMPRESARIAL

Se desarrollará una aplicación web full-stack bajo arquitectura cliente-servidor que centraliza los procesos de stock

Capacidades de Negocio

Las capacidades de negocio se organizan y ventas eliminando el trabajo manual y ofreciendo información en tiempo real al administrador y vendedores

Misión, Visión y Objetivos Estratégicos

Atributo	Descripción
Misión	Modernizar la operación de Ferroalamano digitalizando la gestión de inventario, ventas y cotizaciones para ofrecer un servicio ágil, preciso y transparente a contratistas y compradores.
Visión	Ser la ferretería de referencia en la región, reconocida por su eficiencia operativa digital, respuesta inmediata al cliente y control exacto de su inventario.
Objetivo Estratégico 1	Eliminar los errores humanos en el registro de inventario y ventas mediante automatización.

Objetivo Estratégico 2	Habilitar la generación de cotizaciones formales en formato PDF disponibles fuera del horario físico.
Objetivo Estratégico 3	Proveer al administrador información en tiempo real para la toma de decisiones comerciales y financieras.

en tres niveles de criticidad:

Capacidades Estratégicas	Capacidades de Gestión	Capacidades Operativas
Toma de decisiones basada en datos	Control de inventario en tiempo real	Búsqueda de productos por SKU
Planeación de reabastecimiento	Gestión de clientes y cotizaciones	Registro de ventas y facturación
Monitoreo de desempeño comercial	Administración de usuarios y roles	Generación de PDF de cotizaciones
	Alertas de stock mínimo	Descuento automático de stock

Cadena de Valor del Negocio

La cadena de valor de Ferroalamano con la solución propuesta se estructura de la siguiente forma:

1. Recepción de Mercancía	2. Gestión de stock	3. Atención al Cliente	4. Venta	5. Facturación	6. Análisis y Decisiones
--	--------------------------------	-----------------------------------	-----------------	---------------------------	-------------------------------------

Registro de entrada de productos y actualización de stock.	Control de existencias, alertas de stock mínimo, búsqueda por SKU.	Consulta de productos, precios y disponibilidad desde el sistema.	Creación de venta directa con cálculo automático.	Emisión de factura digital y descuento automático del inventario.	Dashboard con totales vendidos, transacciones y promedio de venta.
--	--	---	---	---	--

CODIFICACIÓN DE MÓDULOS

Arquitectura Monolítica.

La arquitectura monolítica hace que todo el sistema vive en el mismo proyecto, se ejecuta en el mismo servidor y se despliega todo junto. Esto se aplica a mi código ya que se hace proyecto en React corriendo en el puerto 3000, y otro en Node en el puerto 8080, es decir un ambiente de desarrollo. En app.js configuramos Express para que entregue el HTML y también procese el inicio de sesión en el mismo puerto 3000. Es una sola caja que contiene todo el sistema de Ferroallamano.

El Patrón MVC (Modelo - Vista - Controlador)

El patrón MVC hace que dividamos el proyecto en tres grandes capas para que el HTML y las validaciones de base de datos no estén mezclados en el mismo archivo.

VISTA.

Aquí es donde están index.ejs, dashboard.ejs, el CSS y el app.js del cliente. El único trabajo es mostrar información y capturar los clics del usuario. La vista no sabe si la contraseña es correcta, solo sabe enviársela al servidor y mostrar alertas.

MODELO.

Es la clase User. Su único trabajo es definir qué forma tiene un usuario, tiene un email y un password. La clase no sabe de internet, ni de HTML, ni de rutas. Solo representa la estructura de tus datos.

CONTROLADOR.

Es el intermediario. Cuando el usuario hace clic en entrar, la Vista llama a la ruta, y la ruta le pasa el trabajo al Controlador. El Controlador agarra el email y la contraseña, se voltea y le dice al Servicio que revise si los datos son válidos. Cuando el Servicio le responde, el

Controladorva hacia la Vista y le dice que lo deje pasar (aviso 200 OK) o si no muestra contraseña incorrecta(error 401).

Principios SOLID.

El principio dice que un archivo o clase debe tener solo una razón para cambiar. esto se aplica de la siguiente manera:

- router.js: Solo enruta. Si se cambia la URL de login a entrar, solo se toca este archivo.
- authController.js: Solo maneja HTTP.
- authService.js: Solo valida la contraseña. Si Ferroallamano decide que las contraseñas tienen que estar encriptadas, solo se cambia este archivo. El Controlador no sabe de esto.
- userRepository.js: Solo maneja el almacenamiento de datos. Si pasaramos de datos temporales en memoria a MongoDB, solo se tocaría este archivo. El Servicio y el Controlador no tienen que cambiar ni una línea.

Patrón de Diseño: Singleton

El singleton es un patrón que garantiza que una clase tenga un solo objeto creado en toda la aplicación, y nos da un punto de acceso global.

Si no usáramos Singleton, cada vez que el Servicio necesitará buscar un usuario haría un nuevo UserRepository(). Esto crearía una nueva lista y limpia en la memoria RAM del computador cada vez que alguien intenta iniciar sesión, borrando cualquier usuario nuevo que hubieras registrado 5 segundos antes.

DESIGN THINKING

Empatizar

Herramienta utilizada: Análisis Dimensional

Desglosamos la realidad de Ferroalamano evaluando el problema desde sus múltiples dimensiones para comprender el impacto total:

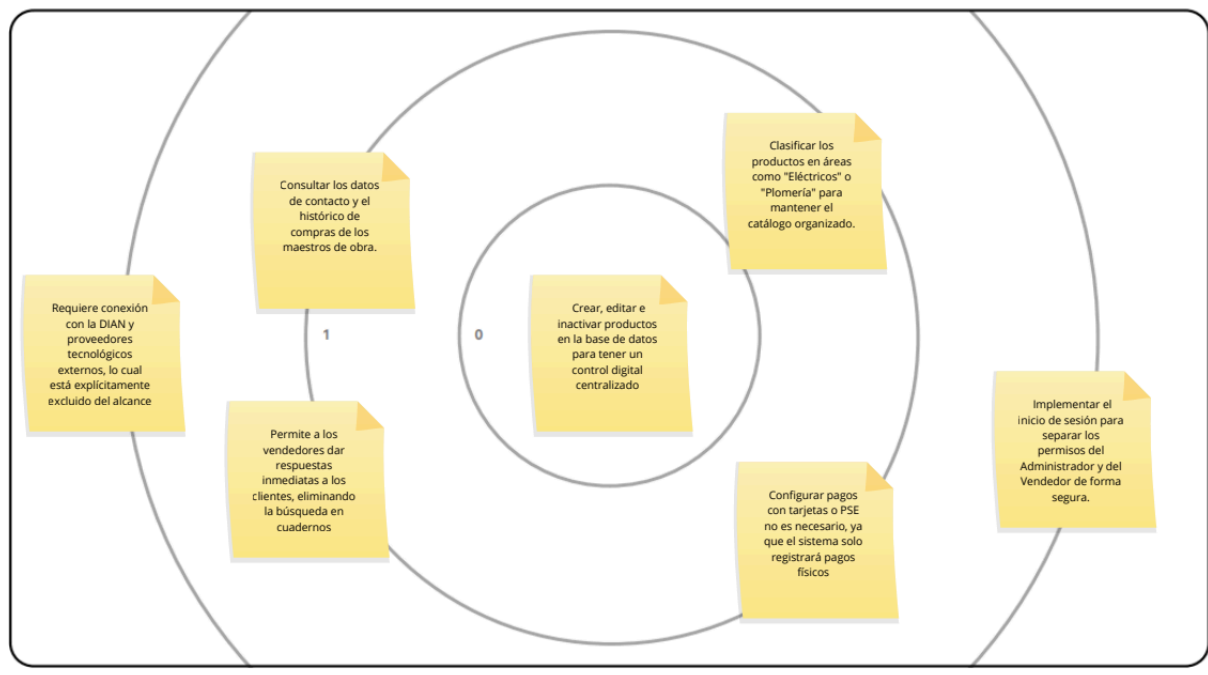
- **Dimensión Operativa:** El negocio no cuenta con un sistema digital; el registro histórico de existencias se lleva en papeles o cuadernos, lo que provoca ventas de productos agotados y desactualización de precios.
- **Dimensión Comercial:** Las cotizaciones se hacen exclusivamente por teléfono o visitas presenciales, limitando el servicio al horario de atención y generando demoras.
- **Dimensión Humana:** El personal sufre una alta carga operativa manual, y los clientes enfrentan problemas para planificar sus proyectos.



Definir

Herramienta utilizada: Diagrama de Prioridades Sintetizamos los hallazgos de la investigación para establecer una jerarquía de necesidades y determinar el alcance crítico del proyecto:

- **Prioridad Operativa:** La urgencia de un control digital centralizado para evitar pérdidas financieras y errores en el manejo del inventario físico.
- **Prioridad Funcional:** El desarrollo de procesos automatizados para la búsqueda por SKU y la generación de cotizaciones que eliminen el uso de papel.
- **Filtro de Alcance:** Se prioriza la seguridad y la trazabilidad interna, postergando integraciones complejas como la facturación electrónica ante la DIAN.



Idear

Herramienta utilizada: Lluvia de Ideas Generamos múltiples soluciones creativas enfocadas en los módulos principales del sistema para transformar la operatividad de Ferroalamano:

- **Módulos de Control:** Creación de alertas de stock mínimo, validación de existencias en tiempo real y registro multicanal de medios de pago.
- **Experiencia de Usuario:** Implementación de un historial de cotizaciones exportables a PDF para facilitar la planificación de proyectos a los contratistas.
- **Estrategia de Valor:** Optimización de los tiempos de respuesta mediante la trazabilidad de transacciones y el acceso a información fuera del horario físico.



Evaluación de Tecnologías con justificación

Para el desarrollo de la aplicación web de Ferroalamano se evaluaron y seleccionaron tecnologías de código abierto, buscando un equilibrio entre facilidad de desarrollo, rendimiento y portabilidad. Se empleó un único lenguaje (JavaScript) a lo largo de toda la solución, lo que reduce la curva de aprendizaje del equipo y agiliza el desarrollo.

Tecnología	Rol en el proyecto	Justificación	Alternativa descartada
Node.js	Entorno de ejecución del backend	Permite JS en cliente y servidor (un solo lenguaje), no bloqueante, ideal para una app de inventario/ventas	PHP, Java (mayor curva/peso)
Express 5	Framework web / enrutamiento y API	Minimalista, middleware flexible, estándar de facto en Node	NestJS (sobredimensionado para el alcance)
EJS	Motor de plantillas (vistas)	Renderiza HTML del lado servidor con datos reales, simple de aprender	React (curva mayor; no se necesita SPA)
MongoDB Atlas + Mongoose	Base de datos NoSQL documental + ODM	Esquema flexible que se adapta a documentos (venta con ítems embebidos); Atlas da BD gestionada en la nube	MySQL (rigidez relacional para snapshots embebidos)
dotenv	Gestión de variables de entorno/secretos	Separa credenciales del código; funciona local y en producción	Hardcodear credenciales (inseguro)
html2pdf.js	Exportar la factura a PDF en el navegador	Genera el PDF sin servidor adicional ni dependencias pesadas	Puppeteer (≈300 MB, excesivo)
Jest	Framework de pruebas	Estándar en JS, soporta cobertura nativa (Istanbul)	Mocha+Chai (más configuración)
ESLint	Análisis estático de código	Detecta errores y malas prácticas antes de ejecutar	JSHint (menos potente)

Render	Plataforma de despliegue	Despliegue automático desde Git, soporta Node + variables de entorno; cumple RQNF004 (portabilidad)	Servidor propio (mantenimiento)
Git/GitHub	Control de versiones	Trabajo colaborativo por ramas (main, Daniel, Jeronimo, test)	—

Gestión de Complejidad: Medición ciclométrica y estrategias

Resumen global: 27 archivos, 1.686 líneas, 71 funciones con $CC \geq 2$, CC promedio ≈ 3.9 , distribución: 55 funciones bajas (<5), 10 moderadas (5–10), 6 altas (>10).

Función	Archivo	CC	Nivel
_validate	productService.js	19	Alto
listProducts	productService.js	16	Alto
createSale	saleService.js	16	Alto
getResumen	dashboardService.js	15	Alto
generarDesdeVenta	facturaService.js	13	Alto
updateProduct	productService.js	11	Alto
createClient / searchClients	clientService.js	7 / 6	Moderado
getUserRole / parseCookie	authMiddleware.js	6 / 6	Moderado

Interpretación: el 77% de las funciones tienen CC bajo (buena mantenibilidad); las 6 funciones altas concentran la lógica de negocio (validaciones y cálculo de ventas/totales).

Estrategias aplicadas/propuestas para reducir complejidad:

- Patrón Repository + Service que separa acceso a datos de la lógica (ya implementado).
- Extraer la validación de _validate a reglas declarativas (tabla de validadores) en vez de muchos if.
- Dividir createSale en submétodos (_validarItems, _calcularTotales).
- Cláusulas de guarda (return temprano) para reducir anidamiento.
- El patrón Decorator (ya documentado) para descuentos/impuestos evita ramas condicionales en el cálculo de totales.

Calidad del Software: Plan de pruebas completo y matriz de resultados

El plan de pruebas tuvo como objetivo verificar que el sistema cumple los requerimientos funcionales y que su lógica de negocio se comporta correctamente ante entradas válidas e inválidas. Se utilizó Jest como framework de pruebas automatizadas, ejecutable con el comando npm test, y se diseñaron los casos combinando tres técnicas complementarias de prueba de software, además de pruebas unitarias:

La técnica de caja blanca examina la lógica interna de cada función conociendo su implementación, asegurando que se recorran sus distintos caminos; se aplicó a los servicios de autenticación, clientes, facturas, ventas y productos. La técnica de caja gris combina conocimiento parcial de la implementación con la verificación de la integración entre el servicio y la capa de datos; se aplicó a clientes, facturas, productos y ventas. La técnica de caja negra evalúa el sistema solo por sus entradas y salidas, sin conocer el código, validando que ante una entrada se produzca la salida esperada; se aplicó a productos y ventas. Finalmente, las pruebas unitarias verificaron de forma aislada los modelos de datos y el middleware de

Técnica	Qué prueba	Casos
Caja blanca (lógica interna de servicios)	authService, clientService, facturaService, ventas, productoService	53
Caja gris (integración servicio+datos)	clientes, facturas, productos, ventas	24
Caja negra (entradas/salidas sin ver código)	productos, ventas	17
Unitarias (modelos y middleware)	models, authMiddleware	15
Total		109

La siguiente matriz resume casos representativos:

ID	Caso de prueba	Entrada	Resultado esperado	Resultado	Estado
TP01	Login credenciales válidas	admin@.../admin1234	success=true, rol=admin	Igual	

TP0 2	Login contraseña incorrecta	admin@.../xxxx	401 "Contraseña incorrecta"	Igual	✓
TP0 3	Crear producto SKU duplicado	sku existente	409 "Ya existe...SKU"	Igual	✓
TP0 4	Venta con stock insuficiente	cantidad > stock	400 "Stock insuficiente"	Igual	✓
TP0 5	Venta con descuento 5%	subtotal 1000, 5%	desc 50, IVA 180.5, total 1130.5	Igual	✓
TP0 6	Generar factura desde venta	ventaId válido	factura borrador FAC-AAAA-NNNN	Igual	✓
TP0 7	Emitir factura	factura borrador	estado=emitida	Igual	✓

Análisis estático de código y métricas de calidad

Cobertura de código (Jest + supertest) — rama test:

Métrica	Cobertura	Cubierto/Total
Sentencias (Statements)	95.75%	248/259
Ramas (Branches)	90.65%	194/214
Funciones (Functions)	93.61%	44/47
Líneas (Lines)	96.62%	229/237

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
all files	95.75	90.65	93.61	96.62	
middlewares	90.47	90	100	94.44	
authMiddleware.js	90.47	90	100	94.44	24
models	100	66.66	100	100	
clientModel.js	100	100	100	100	
facturaModel.js	100	100	100	100	
productModel.js	100	100	100	100	
saleModel.js	100	66.66	100	100	69-70
userModel.js	100	100	100	100	
services	95.1	91.48	90.32	95.75	
authService.js	100	100	100	100	
clientService.js	100	92.3	100	100	58-59
facturaService.js	100	86.11	100	100	39-55
productService.js	98.71	93.47	100	100	6,16,31-35,97-108
saleService.js	84	90	57.14	84.78	8-17,31-33
===== Coverage summary =====					
Statements	: 95.75% (248/259)				
Branches	: 90.65% (194/214)				
Functions	: 93.61% (44/47)				
Lines	: 96.62% (229/237)				
=====					
Test Suites:	13 passed, 13 total				
Tests:	117 passed, 117 total				
Snapshots:	0 total				
Time:	5.055 s				

El sistema alcanza una cobertura del 95.75% en statements, 90.65% en branches, 93.61% en functions y 96.62% en líneas de código, con 117 tests pasados y 0 defectos detectados en la última ejecución.

Los casos documentados en este informe constituyen la evidencia objetiva de que las funcionalidades core del sistema operan conforme a los requisitos establecidos.

Casos de Uso

Las pruebas fueron implementadas con Jest como framework de testing y Supertest para las pruebas de integración HTTP. La estrategia de mocking se limitó estrictamente a las dependencias externas reales (repositorios = capa de BD), preservando la lógica de negocio de los servicios sin alterar. Se realizó la siguiente documentación de Casos de Uso:

Campo	Contenido
ID	TC-AUTH-LOGIN-001
Título	Login exitoso con email válido y contraseña correcta
Módulo	AUTH
Tipo	Caja Blanca — Cobertura de ramas (rama: credenciales correctas)
Prioridad	P1 — CRÍTICO
Requisito / US	US-AUTH-001 · El sistema debe permitir acceso a usuarios registrados
Precondiciones	Usuario admin@ferroallamano.com existe en BD con password admin1234 y rol admin
Datos de entrada	email: admin@ferroallamano.com / password: admin1234
Pasos	1. Llamar authService.authenticate(email, password)2. Verificar respuesta
Resultado esperado	• success: true• message contiene “exitoso”• user contiene { email, role: 'admin' }• user.password es undefined (no expone credencial)
Resultado real	 PASÓ

Campo	Contenido
Estado	Pasó
Automatizable	Sí — Jest (authservice.blanca.test.js)
Defecto	—
TC-AUTH-LOGIN-002 — Usuario no encontrado	
Campo	Contenido
ID	TC-AUTH-LOGIN-002
Título	Login fallido: usuario no existe en base de datos
Módulo	AUTH
Tipo	Caja Blanca — Cobertura de ramas (rama: usuario no encontrado)
Prioridad	P1 — CRÍTICO
Requisito / US	US-AUTH-001
Precondiciones	BD limpia de usuarios con email nadie@x.com
Datos de entrada	email: nadie@x.com / password: loquesea
Pasos	1. Mockear userRepository.findByEmail para que devuelva null 2. Llamar authService.authenticate(email, password) 3. Verificar respuesta
Resultado esperado	• success: false • message contiene “no encontrado” • user es undefined
Resultado real	 PASÓ
Estado	Pasó
Automatizable	Sí — Jest
Defecto	—

TC-AUTH-LOGIN-003 — Contraseña incorrecta

Campo	Contenido
ID	TC-AUTH-LOGIN-003
Título	Login fallido: contraseña no coincide con la registrada
Módulo	AUTH
Tipo	Caja Blanca — Cobertura de ramas (rama: password incorrecto)
Prioridad	P1 — CRÍTICO
Requisito / US	US-AUTH-001
Precondiciones	Usuario admin@ferroallamano.com existe en BD
Datos de entrada	email: admin@ferroallamano.com / password: clave-equivocada
Pasos	1. Mockear usuario existente 2. Llamar authenticate con password erróneo 3. Verificar
Resultado esperado	• success: false • message contiene “contraseña” • user ES undefined
Resultado real	 PASÓ
Estado	Pasó
Automatizable	Sí — Jest
Defecto	—

Para revisar la documentación completa, se recomienda acceder al siguiente link:

https://docs.google.com/document/d/1iH8TV1lm5ecO2vmx2RxHNW48AIGDAwEd_ftnoeJHPmU/edit?usp=sharing

Documentación (técnica y de usuario)

El modelo de datos se compone de las colecciones users, clients, products, sales y facturas, donde la venta embebe sus ítems y una copia de los datos del cliente para garantizar trazabilidad histórica. La comunicación con el frontend se realiza mediante una API que expone, entre otros, los endpoints GET/POST /api/productos, GET/POST /api/ventas, GET

/api/clientes y POST /api/facturas/:id/emitir. La documentación técnica incluye además la guía de instalación y despliegue: clonar el repositorio, ejecutar npm install, configurar el archivo .env con la variable MONGODB_URI, sembrar los datos iniciales con npm run seed y levantar el servidor con npm start, así como el procedimiento de publicación en Render con sus variables de entorno.

Documentación de usuario: <https://youtu.be/QiqJ28nQSB4>

Tendencias Tecnológicas identificadas

Relaciona cada tendencia con tu proyecto:

- Bases de datos en la nube: MongoDB Atlas — sin administrar servidores de BD.
- Despliegue: Render con despliegue automático desde Git.
- NoSQL documental: flexibilidad de esquema frente a lo relacional.
- Renderizado del lado servidor (SSR) y apps responsivas frente a SPA pesadas.
- Generación de documentos en cliente (PDF en el navegador).
- Analítica de negocio(tu dashboard con métricas en tiempo real).
- Tendencias futuras aplicables: facturación electrónica (DIAN), IA para predicción de demanda/stock, lectores de código de barras/SKU.

Design Thinking – Testear (5 usuarios, encuesta, mejoras)

Matriz de resultados (ejemplo):

Usuario	Rol	Tareas completadas	Puntaje SUS (0–100)	Comentario
U1	Admin	4/4	88	"Muy claro el dashboard"
U2	Vendedor	4/4	80	"Rápido para facturar"
U3	Vendedor	3/4	72	"Buscaba el botón de PDF"
U4	vendedor	4/4	85	"Interfaz intuitiva"
U5	vendedor	3/4	75	"Iconos antes confusos"

Conclusiones del Ciclo de Vida del Desarrollo

Fase de Planificación: Se determinó que la viabilidad del proyecto radica en la sustitución de procesos manuales por un desarrollo web centralizado. Esta fase permitió establecer un desarrollo enfocado en resolver las fallas financieras más urgentes, como la pérdida de ingresos por desactualización de precios y la falta de control sobre el stock real de la bodega.

Fase de Análisis: La identificación de las necesidades de los tres actores principales (dueño, vendedores y clientes) fue crucial para definir el alcance técnico. Se concluyó que el sistema no solo debe registrar datos, sino automatizar cálculos y generar documentos, transformando los requerimientos del negocio en funcionalidades de software que reducen la carga operativa.

Fase de Diseño del Sistema: La estructuración bajo el modelo cliente-servidor y el uso de bases de datos garantizan una solución robusta y flexible para la naturaleza de la ferretería. El diseño de los diagramas UML y la interfaz de usuario aseguran que el flujo de información sea lógico y fácil de usar.

Fase de Implementación/Desarrollo: se materializó el diseño en una aplicación funcional construida con Node.js, Express, EJS y MongoDB, organizada en una arquitectura por capas y apoyada en patrones de diseño (Repository, Service y Singleton para la conexión a la base de datos). La medición de la complejidad ciclomática confirmó que la mayor parte del código se mantuvo en niveles bajos, favoreciendo su mantenibilidad.

Fase de Pruebas: se diseñó y ejecutó una estrategia de pruebas con técnicas de caja blanca, gris y negra, además de pruebas unitarias, sumando 109 casos automatizados en Jest y alcanzando una cobertura de sentencias del 95.75%. Esto permitió verificar el cumplimiento de los requerimientos funcionales y detectar errores antes de la entrega.

Fase de Despliegue: la aplicación se publicó en la nube mediante Render, con la base de datos alojada en MongoDB Atlas y la gestión de credenciales a través de variables de entorno, cumpliendo el requisito de portabilidad. Esto convierte al prototipo en un sistema accesible y operativo.

Fase de Mantenimiento (proyección): la combinación de un análisis estático sin hallazgos, una alta cobertura de pruebas y una baja complejidad ciclomática reduce el riesgo de regresiones y facilita la incorporación de nuevas funcionalidades, sentando las bases para la evolución futura del sistema.

REFERENCIAS

- [1] Lim, H. Q., Rahmat, A. R., & Suban@Ani, S. (2025). An inventory management system for small businesses. *Journal of Digital System Development*, 3(1), 29–44.
- [2] Otober, E., & Alegbe, C. (2025). Design of a web-based inventory management system for small and medium-sized production companies. *International Journal of Innovative Information Systems & Technology Research*, 13(1), 127–136.
<https://www.seahipublications.org/wp-content/uploads/2025/01/IJIISTR-M-15-2025.pdf>

- [3] Young, M. N., et al. (2024). Determining user requirements for a tailored web-based inventory and sales management system in micro-small construction supply stores using Design Thinking. En *Proceedings of the 6th World Symposium on Software Engineering (WSSE 2024)*. ACM. <https://doi.org/10.1145/3698062.3698082>
- [4] Martins, J. (2025, 18 febrero). ¿Quiénes son los stakeholders de un proyecto? [2025] • Asana. Asana. <https://asana.com/es/resources/project-stakeholder> 7
- [5] McCabe, T. J. (1976). *A Complexity Measure*. IEEE Transactions on Software Engineering, SE-2(4), 308–320.
- [6] ISO/IEC 25010:2011. *Systems and software engineering — SQuaRE — System and software quality models*.
- [7] Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). Wiley.
- [8] Jest. *Jest Documentation*. <https://jestjs.io/docs/getting-started>
- [9] ESLint. *ESLint - Pluggable JavaScript Linter*. <https://eslint.org/docs/latest/>
- [10] MongoDB, Inc. *MongoDB Atlas Documentation*. <https://www.mongodb.com/docs/atlas/>
- [11] Render. *Render Documentation – Deploy a Node Express App*. <https://render.com/docs>
- [12] Brown, T. (2009). *Change by Design*. Harper Business / IDEO.